

Running snp_call_nf on delgemebioserver

A follow-along guide

Contents

Before we start	2
Part 0 — Getting connected (13:00)	3
0.1 Log in	3
0.1b Start a session that survives	3
0.2 Turn on the software	3
0.3 Check it worked	4
0.4 Make your own working folder	4
Part 1 — Your first run, without running anything (13:10)	5
1.1 The dry run	5
1.2 Read what it printed	5
Part 2 — What actually happened (13:25)	6
2.1 What this pipeline actually does	6
2.2 Process vs. task — the one idea that matters	7
2.2b Now look at the numbers going down the page	8
2.3 The work directory	8
2.4 Where the recipes live	9
2.5 The order — how the processes get connected	11
First: what is a channel?	11
Now: the order	12
2.6 Resume	14
Break — 14:10 to 14:20	14
Part 3 — A real run (14:20)	14
3.1 Launch	14
3.2 While it runs — where the pipeline runs things	15
3.3 When it finishes	15
Which of these you will actually look at	16
Part 4 — Reading the files you just made (14:40)	16
4.1 FASTQ — what went in	16
4.2 BAM — reads placed on the genome	17
4.3 VCF — the variants	18
The columns	18
A cleaner view	19
Counting	19
4.4 One more option — choosing where results go	19

Break — 15:05 to 15:15	20
Part 5 — When it breaks (15:15)	20
5.1 Failure one: a path Nextflow cannot find	20
5.1b Now the one that does not fail	21
What a stub run does and does not tell you	21
5.2 Failure two: a task that dies while running	21
5.2a The error block has four things you need	21
5.2b Go to the work directory	22
5.2c Re-run it by hand	23
5.3 Fix it, and do not start over	24
What just happened	24
Why this matters more than it looks	24
If you want a specific run	25
5.4 The four questions	25
Part 6 — Your own data: start small (15:45)	25
6.1 The three-step ladder	25
6.2 The sample sheet is the control	25
6.3 Always stub first	26
6.4 Then measure	26
6.5 Getting real data	27
6.6 Running it somewhere else	27
The full recipe	27
Housekeeping	28
Appendix A — Optional: one option, more parallelism	28
Other options that exist	29
Command reference card	29
Words we used	30
After today	30
Further reading	30

About this version. This guide was delivered on 29 July 2026 and was written against `snpcall_nf` on the main branch at commit 5386e4a. Some behaviour described in Part 5 (the `--parasite_reads_only` failure) and the early-exit options in Part 6 and Appendix A was fixed upstream afterwards (PR #22). To reproduce the session exactly, check out that commit. Hostnames and server paths are placeholders; your instructor will give you the real ones.

Before we start

Everything you need is already installed on the server. You will not install anything today. You will not download anything today. You will not use your own sequencing data today.

You will run a real pipeline on a small test dataset that comes with the pipeline, and you will learn enough to run it on your own data afterwards.

By 16:00 you will be able to:

1. Start Nextflow on this server

2. Run the `snpcall_nf` pipeline
3. Explain what a process, a task, and a channel are
4. Read the pipeline's output and find where results are written
5. Find out *why* a run failed, and restart it without losing finished work
6. Change what the pipeline does using command-line options, without editing code

Rules for today:

- If a command does not work, say so immediately. Do not wait.
- Paste the error in the chat. Errors are the interesting part.
- If you fall behind, say so. We will wait.

Part 0 — Getting connected (13:00)

0.1 Log in

`ssh YOUR_USERNAME@YOUR_SERVER`

0.1b Start a session that survives

Do this before anything else. If your internet drops while the pipeline is running, the pipeline dies with it — unless you start it inside `tmux`.

`tmux new -s nf`

✓ **A green bar appears at the bottom of your terminal.** That means you are inside `tmux`. Everything you run from now on keeps running even if you disconnect.

Two things to remember:

To do this	Press or type
Leave it running and step away	<code>Ctrl-b</code> , release, then <code>d</code>
Come back to it	<code>tmux attach -t nf</code>

`Ctrl-b` then `d` is “detach”. Your work keeps going without you.

If you lose your connection at any point today: log back in with `ssh`, then `tmux attach -t nf`, and you will find everything exactly as you left it — including a pipeline that never stopped running.

Everything after this point assumes you are inside `tmux`. If you ever open a fresh terminal, you will need to attach again *and* re-run the source command below.

0.2 Turn on the software

`source SHARED_TRAINING_DIR/env.sh`

SHARED_TRAINING_DIR is the shared training folder your instructor set up; they will give you the exact path.

This one line points your shell at software that is already installed in a shared folder. It does three things: it clears two broken system settings that would stop Nextflow from starting, it switches on the environment manager, and it activates the environment that contains Nextflow.

There is a **second** environment — the one holding bowtie2, samtools and GATK, the tools the pipeline actually runs. You do not activate that one. Nextflow picks it up on its own, for each step, from a shared copy that is already built.

You do not need to understand this today. You need to remember that you run the source line **every time you open a new terminal**.

0.3 Check it worked

```
nextflow -version
echo $SNPTRAIN
```

✓ **You should see:** version 26.04.6, and a path printed on the last line.

\$SNPTRAIN is a shortcut to the shared training folder. You will use it several times today, so if that last line is empty, tell me now — nothing after this point will work.

If you see anything else — especially an error mentioning java — stop and tell me.

0.4 Make your own working folder

```
mkdir -p $HOME/snp_call_test && cd $HOME/snp_call_test
```

Now copy the pipeline. **Do not copy the whole folder** — the ref/ folder holds the human and parasite reference genomes and is about 16 GB. Eight of us copying that is 128 GB of identical files. We copy the code and *link* to the shared reference instead:

```
rsync -a --exclude='ref' --exclude='.git' --exclude='work' --exclude='results' \
    --exclude='.nextflow*' \
    "$SNPTRAIN/snp_call_nf/" ./snp_call_nf/
ln -s "$SNPTRAIN/snp_call_nf/ref" ./snp_call_nf/ref
cd snp_call_nf
ls -l
```

✓ **You should see:** main.nf, nextflow.config, test_data/, fastq_map.tsv — and ref shown with an arrow -> pointing at the shared folder.

Everybody now has their own private copy of the *code*. Nothing you do can affect anyone else. The reference genomes are shared and read-only, which is fine — nobody needs to write to a reference.

Remember that arrow. You will see the same trick again in Part 2, and for the same reason: Nextflow avoids copying large files whenever it can.

✓ **Your first run should show every step actually running.** If any step says cached before you have run anything, stop and tell me — something was copied that should not have been.

Part 1 — Your first run, without running anything (13:10)

1.1 The dry run

Check you are in the right place first. The pipeline lives one level down from the folder you created:

```
pwd
ls main.nf
```

✓ You should be in `.../snpcall_test/snp_call_nf` and `main.nf` should exist. If not, `cd snp_call_nf`.

```
nextflow main.nf -stub-run
```

Watch the screen. It will finish in seconds.

You will see a yellow line near the top:

```
WARN: Static typing is a preview feature -- syntax and behavior may change...
```

Ignore it. You will see it on every command you run today. It is Nextflow telling us about one of its own new features. It is not about your run and nothing is wrong.

`-stub-run` means: **do everything except actually run the tools.** Nextflow works out the whole plan, creates every folder, and pretends each step succeeded. Nothing is aligned. No variants are called. It costs nothing.

This is the single most useful command in this whole course. It answers “will this pipeline start correctly?” without waiting for GATK.

1.2 Read what it printed

First, the top of the output. Before anything runs, the pipeline prints its own settings:

```
===== PARAMETERS =====
fq_map: .../fastq_map.tsv
split:  chromosomes
filtering:
    hard:    true
    vqsr:    false
parasite_reads_only:  false
coverage_only:  false
gvcf_only:      false
=====
```

Read this every time. It is the pipeline telling you exactly what it thinks you asked for. Most “why did it do that?” questions are answered here.

Then the task table. Each line looks like this:

```
[61/598cb1] BOWTIE2_ALIGN_TO_HOST (Sample02~r1) [100%] 6 of 6 ✓
```

Read it left to right:

Piece	Meaning
[61/598cb1]	The folder where this work happened
BOWTIE2_ALIGN_TO_HOST (Sample02~r1)	The process — the name of the step Sample02, sequencing run r1 — the last one to finish
6 of 6	This step ran 6 times

And above the table:

```
executor > local (115)
```

✓ **115 tasks.** Fifteen steps. Five samples. Remember that number — it comes back when we look at how the work is divided.

local means the tasks ran on this machine. Remember that word too.

Part 2 — What actually happened (13:25)

2.1 What this pipeline actually does

Open the workflow chart in the README (I will share my screen): <https://github.com/bgao068/s>
— scroll to “Workflow Chart”.

It shows every process, which output folder each one writes to, the two alternative routes through host removal, and the three points where you can stop early. Keep it open — we will come back to it.

Scroll back up to your task table. **That list, in that order, is the pipeline.** It follows GATK best practices for germline short variant discovery, adapted for *Plasmodium* by the MalariaGEN Pf6 methods.

Six stages. Each one exists for a reason worth knowing.

1. Remove human reads — BOWTIE2_ALIGN_TO_HOST, SAMTOOLS_VIEW_RM_HOST_READS, SAMTOOLS_FASTQ

Your samples come from patient blood. Most of the DNA in that tube is *human*, not parasite — often the large majority of it. So the pipeline aligns everything to the human GRCh38 genome first and **throws away whatever matches**. What is left is enriched for parasite.

This step is specific to malaria and to clinical samples. A pipeline for a cultured organism would not have it.

2. Align to the parasite — BOWTIE2_ALIGN_TO_PARASITE

The surviving reads are mapped to *P. falciparum* 3D7, PlasmoDB v44.

3. Make the alignments trustworthy — PICARD_MERGE_SORT_BAMS, PICARD_MARK_DUPLICATES, GATK_BASE_RECALIBRATOR, GATK_APPLY_BQSR

Merge the runs belonging to one sample. Flag PCR duplicates so the same original molecule is not counted many times. Then recalibrate the base quality scores — the sequencer’s own confidence estimates are systematically biased, and GATK corrects them against a set of variants already known to be real. Those known variants come from the Pf genetic crosses.

Garbage base qualities produce garbage variant calls. This stage is why the calls at the end can be believed.

4. Check the data — BEDTOOLS_GENOMEcov, SAMTOOLS_FLAGSTAT

Coverage and alignment statistics. This is your QC.

5. Call variants, one sample at a time — GATK_HAPLOTYPE_CALLER

Produces a gVCF per sample: not just “here is a variant”, but a record of *every* position and how confident we are about it, including the boring ones.

6. Call variants across all samples together — GATK_GENOMICS_DB_IMPORT, GATK_GENOTYPE_GVCFS, then GATK_SELECT_VARIANTS and filtration

This is the point of the whole pipeline. If you called each sample independently, a position with no coverage in Sample01 and a variant in Sample02 would look the same as a position that is genuinely reference in Sample01. Joint calling uses evidence from every sample at every position, so you can tell “reference” apart from “we don’t know”. For population genetics — allele frequencies, relatedness, selection — that distinction is everything.

That is why the gVCFs come first and the joint call comes second. It is also why the task count changes from 5 to 14 at this point: joint calling stops being a per-sample operation.

2.2 Process vs. task — the one idea that matters

A **process** is a *recipe*. It is written once.

A **task** is one *cooking* of that recipe. If you have 6 sequencing runs, the host-alignment process produces 6 tasks. Nextflow runs them at the same time, on different CPUs, without you asking.

That is the entire reason to use Nextflow instead of a shell script. You wrote one recipe. You got 115 tasks.

2.2b Now look at the numbers going down the page

Go back to your task table and read only the counts, from top to bottom:

BOWTIE2_ALIGN_TO_HOST	6 of 6	
SAMTOOLS_VIEW_RM_HOST_READS	6 of 6	
SAMTOOLS_FASTQ	6 of 6	
BOWTIE2_ALIGN_TO_PARASITE	6 of 6	
PICARD_MERGE_SORT_BAMS	5 of 5	<-- changed
PICARD_MARK_DUPLICATES	5 of 5	
GATK_BASE_RECALIBRATOR	5 of 5	
GATK_APPLY_BQSR	5 of 5	
GATK_HAPLOTYPE_CALLER	5 of 5	
GATK_GENOMICS_DB_IMPORT	14 of 14	<-- changed again
GATK_GENOTYPE_GVCFS	14 of 14	
GATK_SELECT_VARIANTS	14 of 14	
GATK_VARIANT_FILTRATION	14 of 14	

6, then 5, then 14. Why?

Look at the sample sheet:

```
tail -n +2 fastq_map.tsv | cut -f1 | sort -u
```

There are **5 samples** — but **6 sequencing runs**, because Sample10 was sequenced twice (Sample10~r1 and Sample10~r2). The early steps work on *runs*, so they run 6 times.

PICARD_MERGE_SORT_BAMS combines Sample10's two runs into one BAM. From that point on the unit of work is a **sample**, so: 5.

Then joint calling arrives, and the work is no longer divided by sample at all — it is divided by **chromosome**. *P. falciparum* has 14. So: 14.

This is what a channel does. A channel is the queue of items flowing between two steps, and a **process runs once for every item in it**. Six items, six tasks.

So the counts changed because the *items* changed — six runs were grouped into five samples, and then the work was re-cut into fourteen chromosomes. The number of tasks is not a setting you chose. It is a consequence of how the data is grouped at each stage.

We will see the exact lines that do this in 2.5. If you understand this, you understand Nextflow.

2.3 The work directory

Every task gets its own private folder. Look:

```
ls work/
```

Then go into one of them. Use the code from your own screen output — the [a1/3f8c2d] part is the folder name, split by the /:

```
cd work/a1/3f8c2d*
ls -la
```

✓ **You should see** several hidden files starting with `.command`

Open the most important one:

```
cat .command.sh
```

This is the exact command that was run. Not a description of it. The real thing. You can copy this and run it by hand.

Two others you will need later:

```
cat .command.err  # error messages from the tool
cat .command.log  # everything the tool printed
```

Also notice: the input files in this folder are **symbolic links** (`ls -la` shows them with an arrow `->`). Nextflow did not copy your data. It linked it. That is why this is fast and does not fill the disk.

Go back:

```
cd $HOME/snpccall_test/snp_call_nf
```

2.4 Where the recipes live

Every process you saw in the task table is written down in `main.nf`. Find them:

```
grep -n "^process" main.nf
```

✓ **You should see the same names that appeared in your run** — the list on your screen a moment ago is the list in the file.

Count them. There are 18. But your run had 15 steps.

Three processes are written in the file and did not run:

Process	Why it was skipped
BOWTIE2_ALIGN_TO_CONCAT_GENOME	use_concat_genome: false
GATK_VARIANT_RECALIBRATOR	vqsr: false
GATK_APPLY_VQSR	vqsr: false

Check the PARAMETERS banner from your run — all three settings are printed there.

This is the most important idea in the course. `main.nf` is not a fixed sequence of steps. It is a set of available steps, and your parameters decide which ones are used. You change what the pipeline *does* without changing what the pipeline *is*.

That is also why `results/vqsrfilt_vcf` was empty. The recipe exists. You did not order it.

Now open the shortest one:

```
sed -n '364,381p' main.nf
```

```

process SAMTOOLS_FLAGSTAT {
    tag "${sample}"

    input:
    tuple(sample: String, bam: Path)

    output:
    tuple(sample, file("*.flagstat"))

    script:
    """
    samtools flagstat ${bam} > ${sample}.flagstat
    """

    stub:
    """ touch ${sample}.flagstat """
}

```

Eighteen lines. Go through it slowly — every part connects to something you have already seen today.

process SAMTOOLS_FLAGSTAT {

The name. This is exactly the name that appeared in your task table.

tag "\${sample}"

The label. Remember the task table line SAMTOOLS_FLAGSTAT (Sample09) — the part in parentheses came from here. With five samples you get five tasks, and the tag is how you tell them apart on screen and in the logs. Without a tag you would just see five identical lines.

input: — tuple(sample: String, bam: Path)

What is a tuple? A group of items that travel together as one unit, in a fixed order. You already know what one looks like — a line of your sample sheet:

```
Sample01    0    r1    1    test_data/fastq/Sample01_1.fastq.gz
```

Five fields that belong together. Separate them and each becomes meaningless: a path with no sample name, a sample name with no file.

That is the problem a tuple solves. A channel carries items from one step to the next. If it carried only BAM files, the pipeline would have five files and no way to tell whose is whose. By carrying (sample, bam) as a pair, **the label never gets separated from the data.**

So this line reads: *this step needs a sample name and a BAM file, arriving together.* sample: String is text, bam: Path is a file.

This is also where the task table gets its labels. SAMTOOLS_FLAGSTAT (Sample09) — the name was in the tuple alongside the BAM.

Path is not just text. When Nextflow sees a Path input it **stages the file into the task's work directory** — that is where the symlinks you saw in 2.3 come from.

This typed style — `sample: String, bam: Path` — is a new Nextflow feature. It is the reason for the yellow Static typing is a preview feature warning on every run.

output: — `tuple(sample, file("*.flagstat"))`

What to keep — and note it is a tuple again. `*.flagstat` is a pattern, not a filename: after the script finishes, Nextflow looks in the work directory for anything matching and captures it. **The sample name is passed along with it**, so the next step still knows whose file this is.

Tuple in, tuple out. The label rides along the whole length of the pipeline, which is how a variant call at the very end can still be traced to the sample it came from.

Notice the process does not say where the file goes. It does not know about results/, and it does not know what runs next. It only declares what it needs and what it produces.

script:

The actual command, between triple quotes. `${bam}` and `${sample}` are filled in for each task before it runs.

This is the same text you read in .command.sh in 2.3 — except there the variables had already been substituted. Here is the template; there was the result.

stub:

A fake version of the script. `touch` creates an empty file with the right name and nothing else.

This is why Part 1 worked. When you ran `-stub-run`, Nextflow ran this block instead of the real one. No samtools, no BAM reading — just an empty file, so the next step has something to receive. That is how the whole pipeline completed in thirteen seconds.

Stub blocks are optional, and a pipeline without them cannot be dry-run. Bing wrote one for every process here, which is a kindness to everyone who uses it.

Every one of the eighteen processes in this file has this shape. Different tools, different inputs, same six parts. Once you can read this one, you can read all of them — and more importantly, you can read processes in pipelines you have never seen before.

2.5 The order — how the processes get connected

First: what is a channel?

We have used the word twice. Here is what it means.

A **channel** is the connection between two processes — a queue that carries items from one to the next. Each item is usually a tuple: a sample name and a file, travelling together.

Three rules, and everything else follows from them:

1. **A process reads from an input channel and writes to an output channel.**
2. **A process runs once for every item in its input channel.** Six items, six tasks. This is where your task counts come from.
3. **A channel can be reshaped before it reaches the next process** — grouped, filtered, joined, or built from something else entirely. That is how six items become five, and how five become fourteen.

A process does not read files from a directory and it does not know what came before it. It receives items from a channel, one at a time, and emits items to another. Nothing more.

Now: the order

The processes do not know about each other. So how does Nextflow know what runs after what?

```
grep -n "^workflow" main.nf
```

One line — 575:workflow {. That is where it starts; it runs to the end of the file. To browse it:

```
less +575 main.nf
```

(q to quit, space to page down.)

It is about 200 lines, and most of that is file-path handling. **You do not need to read it all.** One command shows you the part that matters:

```
grep -n "out_.* = [A-Z]" main.nf
```

✓ **Every process invocation in the pipeline, in order.** That short list is the whole assembly.

The pattern. Look at three consecutive lines from it:

```
out_BOWTIE2_ALIGN_TO_HOST      = BOWTIE2_ALIGN_TO_HOST(input_ch)
out_SAMTOOLS_VIEW_RM_HOST_READS =
  ↪ SAMTOOLS_VIEW_RM_HOST_READS(out_BOWTIE2_ALIGN_TO_HOST...)
out_SAMTOOLS_FASTQ              = SAMTOOLS_FASTQ(out_SAMTOOLS_VIEW_RM_HOST_READS)
```

Read them as a sentence: align to host, then hand that result to the read removal, then hand *that* to the FASTQ conversion. **The output of one process is the input of the next.** That is the whole idea. Everything else in this block is detail.

You never write “step 3 comes after step 2”. You write which channel goes where, and the order follows.

Now five specific lines, each of which explains something you saw earlier today.

1 • Where your sample sheet becomes a channel

```
grep -n -A6 "splitCsv" main.nf | head -12
```

```
.flatMap { csv -> csv.splitCsv(skip: 1, sep: '\t') }
```

skip: 1 — ignore the first line, because it is the header. sep: '\t' — split on tabs.

This is why your sample sheet must use tabs and must have a header. Not a style rule. This line.

2 • Where 6 became 5

```
grep -n "merge_input" main.nf
```

```
merge_input = out_BOWTIE2_ALIGN_TO_PARASITE.map { ... }.join(n_run, by:
  ↪ "sample").groupBy()
```

```
out_PICARD_MERGE_SORT_BAMS = PICARD_MERGE_SORT_BAMS(merge_input)
```

.groupBy() gathers the items by sample. Six runs go in; five groups come out, because Sample10's two runs land in one group. **That is the line that changed your task count from 6 to 5.**

3 • Where 5 became 14

```
grep -n -A1 "interval_ch = channel" main.nf
```

```
interval_ch = channel.fromList(params.genome_intervals[params.split])
```

```
out_GATK_GENOMICS_DB_IMPORT = GATK_GENOMICS_DB_IMPORT(interval_ch, gvcf_map_ch)
```

This channel is not made from samples at all. It is made from the **interval list** in the configuration — 14 chromosomes. So the process runs 14 times. The unit of work changed from a sample to a piece of genome, and this is where.

4 • Where the unused processes were skipped

```
grep -n "if (params" main.nf
```

Several blocks look like this:

```
if (params.vqsr) {
  out_GATK_VARIANT_RECALIBRATOR = GATK_VARIANT_RECALIBRATOR(...)
  out_GATK_APPLY_VQSR = GATK_APPLY_VQSR(...)
}
```

vqsr was false, so this block never ran. **This is why 18 processes were defined and only 15 executed.** The parameters do not delete the processes — they decide which ones get connected.

5 • Where the results folders come from

```
sed -n '/^    publish:/$p' main.nf
```

```
publish:
rp_flagstat_raw      = rp_flagstat_raw
rp_recal_bam         = rp_recal_bam
```

```
rp_hardfilt_vcf      = rp_hardfilt_vcf
...
```

Every name here became a folder in `results/`. That is the connection between the workflow and the output directory you will look at after the run.

You do not need to be able to write this block. You need to know it exists, that it is where the order lives, and that when you want to know why something ran the way it did, the answer is here.

2.6 Resume

Run exactly the same command again, with one flag added:

```
nextflow main.nf -stub-run -resume
```

✓ **You should see:** every line ends in `cached`

Nothing ran. Nextflow recognised that nothing changed and reused all the previous work. This is not a small convenience. When a 40-hour run dies at hour 38, you fix the problem and add `-resume`, and you lose only the failed step.

`-resume` is the most valuable flag in Nextflow. Remember it.

Break — 14:10 to 14:20

Part 3 — A real run (14:20)

Now we run the tools for real. Same test data, same pipeline, but this time `bowtie2` and `samtools` actually execute.

3.1 Launch

```
nextflow main.nf -c $SNPTRAIN/training.config
```

This takes about 16 minutes. Leave it running — we will talk while it works.

`-c $SNPTRAIN/training.config` adds a configuration file that limits how many CPUs each of you can use. There are eight of us and one machine, and this server has no scheduler to keep us apart, so the limits are the only thing preventing us from fighting over the CPUs.

Leave this running. It takes about sixteen minutes.

Your terminal is now busy and will stay busy. You cannot type into it until the run finishes, and you do not need to. Watch the task table — the counts climb and new steps appear as the pipeline works through them. I will talk while it runs.

Do not close the window and do not press Ctrl-c.

Because you are inside tmux, this run is safe. If your connection drops, log back in and `tmux attach -t nf` — you will find it still going.

3.2 While it runs — where the pipeline runs things

Watch the task table on your screen. It updates as tasks finish — counts climb, new steps appear. That is eight people's work happening on one machine.

Nextflow can send work to many places: a Slurm cluster, an SGE cluster, the cloud, or just the machine you are sitting on. That choice is called the **executor**.

Look at the bottom of `nextflow.config`:

```
grep -n "profile" -A 5 nextflow.config
```

You will see `sgs` and `slurm` profiles. Bing wrote those for the servers he uses. If you were on one of those machines you would add `-profile slurm` and every task would be submitted as a cluster job instead.

This server has Slurm installed, but it is not running. So we use the default — the local executor, which just runs things directly on this machine. Same pipeline. Same code. Different place. That is what a profile is for.

This is also why `training.config` matters here and would not matter on a real cluster: on a cluster, the scheduler enforces fairness. Here, we do.

3.3 When it finishes

```
ls results/
```

✓ **You should see** several folders. Look inside a couple of them.

The output folders you will care about most in real work:

Folder	Contents
<code>results/readlen_raw</code>	Raw read length per run
<code>results/flagstat_raw</code>	Alignment stats before host removal
<code>results/parasite_reads</code>	FASTQ files with human reads stripped out
<code>results/flagstat_parasite</code>	Alignment stats against the parasite genome
<code>results/recalibrated</code>	Analysis-ready BAM files
<code>results/recal_bam_coverage</code>	Read depth per sample
<code>results/recal_bam_flagstat</code>	Stats on the final BAMs
<code>results/gvcf</code>	Per-sample variant calls
<code>results/jointcall_vcf</code>	Joint-called VCF, one per chromosome
<code>results/hardfilt_vcf</code>	Hard-filtered VCF
<code>results/vqsrfilt_vcf</code>	VQSR-filtered VCF — empty , because <code>vqsr: false</code>

Which of these you will actually look at

flagstat_raw and flagstat_parasite — read these first, every time. Together they tell you how much of your sample was human and how much was parasite. A sample that is 98% human has very little parasite DNA, and no amount of downstream processing will rescue it. **This is your go/no-go number**, and it is available before you have spent a single CPU-hour on GATK.

recal_bam_coverage — depth across the genome. Low coverage means low confidence calls. The pipeline computes this over the autosomes only (genome_size_bp and chrom_reg in nextflow.config).

gvcf — per-sample calls. Useful if you later want to add samples to a cohort without re-processing everything.

hardfilt_vcf — your final call set for most purposes.

And the empty one. vqsrfilt_vcf exists but is empty, because the PARAMETERS banner said vqsr: false. Nothing failed — you did not order it.

There are two ways to filter variants and the pipeline supports both. **Hard filtering** applies fixed thresholds — QD below 2, FS above 60, MQ below 40 — the same cutoffs for everybody. **VQSR** trains a model on variants already known to be real and learns where to draw the line for *your* data. VQSR is better when it works, but it needs enough variants to train on. With five test samples it has too little to learn from and will crash, which is why it is off by default.

For a real cohort of a few hundred isolates, --vqsr true is worth trying.

Note that results/ is separate from work/. work/ is Nextflow's scratch space and it gets very large. results/ is what you keep.

Part 4 — Reading the files you just made (14:40)

A pipeline is only useful if you can read what comes out of it. Three formats matter, and you will meet all three for the rest of your career.

The tools live in the shared environment. Point at them once:

```
export TOOLS=$SNPTRAIN/envs/snp_call_nf/bin
$TOOLS/samtools --version | head -1
```

4.1 FASTQ — what went in

```
zcat test_data/fastq/Sample01_1.fastq.gz | head -8
```

✓ **Eight lines. That is two reads.** Every read is exactly four lines:

Line	Contents
1	@ then the read name
2	The bases — A, C, G, T, N

Line	Contents
3	+ (a separator, usually nothing else)
4	Quality — one character per base

Line 4 is the same length as line 2. That is the rule worth remembering. Each character encodes how confident the sequencer was about that one base.

```
zcat test_data/fastq/Sample01_1.fastq.gz | wc -l
```

Divide by 4 to get the number of reads.

4.2 BAM — reads placed on the genome

A BAM is a compressed, indexed table of reads *after* alignment. It is binary, so you need a tool to read it.

The header first. A BAM carries a record of where it came from.

```
export BAM=$SNPTRAIN/results_complete/recalibrated/Sample01_recal.bam
$TOOLS/samtools view -H $BAM | grep "^@SQ" | wc -l
$TOOLS/samtools view -H $BAM | grep "^@SQ" | head -3
```

✓ **16 sequences.** Fourteen chromosomes, plus Pf3D7_API_v3 (the apicoplast) and Pf_M76611 (the mitochondrion), with their lengths.

But your joint calling ran **14** tasks, not 16. The organelle genomes were left out — a decision written into `genome_intervals` in `nextflow.config`. The reference has 16 sequences; somebody chose to call variants on 14 of them.

```
$TOOLS/samtools view -H $BAM | grep "^@RG"
```

The **read group**: which sample these reads belong to, and what sequenced them. This is how GATK knows Sample01's reads are Sample01's — without it, joint calling could not work.

```
$TOOLS/samtools view -H $BAM | grep "^@PG" | cut -f2
```

✓ **This is the pipeline's history, recorded inside the file.** bowtie2, then samtools, then MarkDuplicates, then ApplyBQSR — the same steps you watched run this morning, in the same order, permanently attached to the data.

Months from now, when you cannot remember how a BAM was made, this tells you.

Now the reads themselves.

```
$TOOLS/samtools view $BAM | head -2
```

Eleven mandatory columns. Five are worth knowing now:

Column	Meaning
1	Read name — the same name from the FASTQ
3	Which chromosome it aligned to

Column	Meaning
4	Position on that chromosome
5	MAPQ — confidence in the <i>placement</i>
6	CIGAR — how it aligned, e.g. 100M = 100 bases matched

Column 1 is the point. That is the same read you saw in the FASTQ, now with an address on the genome. That is all alignment is.

```
$TOOLS/samtools flagstat $BAM
```

The same kind of summary as the flagstat folders in your results.

4.3 VCF — the variants

This is the file your analysis actually uses.

```
export VCF=$SNPTRAIN/results_complete/hardfilt_vcf/Pf3D7_01_v3.hardfilt.vcf
head -30 $VCF
```

Lines beginning `##` are **metadata** — the file describing itself. Look at one group in particular:

```
grep "^##FILTER" $VCF
```

✓ **You should recognise these.** `QD.lt.2`, `FS.gt.60`, `MQ.lt.40` — these are the hard filters from `nextflow.config`, the ones we looked at earlier. The settings you read in the configuration are written into the output file.

The columns

```
grep -v "^##" $VCF | head -3
```

The first line starts with a single `#` — it is the column header.

Column	Meaning
CHROM	Chromosome
POS	Position
REF	The base in the reference genome
ALT	The base seen instead
QUAL	Confidence that something is here
FILTER	PASS, or which filter it failed
INFO	Summary statistics across all samples
FORMAT + sample columns	The genotype call for each sample

FILTER is where your configuration shows up in your data. A variant marked `QD.lt.2` failed the quality-by-depth threshold set in a config file you can open and change.

A cleaner view

Those lines are wide and hard to read. bcftools is built for this:

```
bcftools view -H $VCF | head -3
```

-H drops the whole ## header and shows only the records.

Better still — ask for exactly the columns you want:

```
bcftools query -f '%CHROM\t%POS\t%REF\t%ALT\t%FILTER\n' $VCF | head -10
```

✓ **Chromosome, position, reference base, alternate base, and whether it passed** — five columns, aligned, nothing else. This is how you actually read a VCF.

The % names are the column names from the table above. Ask for the ones you need, in the order you want them.

Counting

```
bcftools stats $VCF | grep "^SN"
```

Number of records, SNPs, indels, and more — a summary of the whole file in a few lines.

You can also do it with plain shell tools, which is worth knowing for when bcftools is not available:

```
grep -vc "^#" $VCF  
grep -v "^#" $VCF | cut -f7 | sort | uniq -c
```

How many variants, and how many passed versus failed each filter.

You do not need to memorise these formats. You need to know that they are plain, documented, and readable, and where to look them up. The specifications are linked at the end of this guide.

4.4 One more option — choosing where results go

Remember the publish: block at the end of the workflow? Those names became the folders in results/. But results/ itself is not written into the pipeline — it is just the default, and you can change it from the command line.

```
nextflow main.nf -stub-run -output-dir results_demo  
ls results_demo/
```

✓ **Thirteen seconds, and a complete second results tree appears** — same folder names, in a directory you chose.

We used -stub-run so this costs nothing. The files inside are empty, but the structure is real.

```
ls results/ results_demo/
```

Same names. Different place.

Why you will want this. When you run a pilot set and then a bigger set, both write to `results/` by default and the second run overwrites the first. Give each run its own output directory and you can compare them:

```
nextflow main.nf -c $SNPTRAIN/training.config --fq_map pilot.tsv -output-dir
  ↪ results_pilot
nextflow main.nf -c $SNPTRAIN/training.config --fq_map all.tsv -output-dir
  ↪ results_full
```

Two runs, two result sets, nothing lost.

Notice what you did not do. You did not edit `main.nf`. You did not edit `nextflow.config`. You changed where a pipeline writes its output by adding six words to a command — the same way you changed how much of the machine it used, and the same way you will recover a failed run in a moment.

Break — 15:05 to 15:15

Part 5 — When it breaks (15:15)

I am going to break the pipeline on purpose, twice. Follow along. Nothing here is a test and nothing here is a trick — these are the two failures you are most likely to meet in real work.

One thing to know first. Bing wrote this pipeline so that a failing step retries five times and is then **skipped**, letting the rest of the run continue. For a two-day production run over hundreds of samples, that is the right choice — one bad sample should not destroy the whole job.

Today we want the opposite. `training.config` overrides that setting so failures stop the run and print an error. That is why the commands below use `-c $SNPTRAIN/training.config`.

Same pipeline. Same code. Different behaviour, chosen from outside.

5.1 Failure one: a path Nextflow cannot find

```
nextflow main.nf -stub-run --fq_map does_not_exist.tsv
```

✓ **It fails instantly** — before the parameters banner, before a single process is created. Nextflow could not read your sample sheet, so there was nothing to build a pipeline from.

A wrong path costs you five seconds, not five hours.

5.1b Now the one that does not fail

```
cp fastq_map.tsv broken_map.tsv
sed -i 's|test_data/fastq|test_data/fastqq|' broken_map.tsv
head -2 broken_map.tsv
```

Every FASTQ path in that file is now wrong — fastqq instead of fastq. Run it:

```
nextflow main.nf -stub-run --fq_map broken_map.tsv
```

✓ **It succeeds.** All 115 tasks, all green.

Why? A stub run does not read your data. It runs a *fake* version of each step — remember the `stub:` block in 2.4, the one that just did touch. The fake version never opens a FASTQ file, so nothing ever notices the paths are broken.

What a stub run does and does not tell you

A stub run will catch	A stub run will not catch
A sample sheet you cannot find	Data files that do not exist
A sample sheet that will not parse	A corrupt or truncated file
Wrong number of columns	A tool that fails on your data
Unexpected task counts	Anything that needs the tool to run

Use it for the shape of your run, not the health of your data. It is still the cheapest ten seconds you will ever spend — just do not mistake a green stub run for a guarantee.

5.2 Failure two: a task that dies while running

This time the pipeline starts correctly, runs for a while, and then one step fails. This is the harder kind, and the one worth practising.

We are going to use an option that looks harmless and is not:

```
nextflow main.nf -c $SNPTRAIN/training.config --parasite_reads_only
```

`--parasite_reads_only` sounds like it should stop the pipeline after human reads are removed. It does not stop the later steps from being scheduled — so joint calling runs anyway, with nothing to work on, and GATK fails.

Watch what happens. Some steps show green ticks. One turns red, and the run stops.

That is realistic. Not every option does what its name suggests, and the way you find out is by running it.

5.2a The error block has four things you need

Nextflow prints a red block. Ignore its length. Find these four lines:

Caused by:

Process `PROCESS\_NAME (Sample01~r1)` terminated with an error exit status (1)

Command exit status:

1

Command error:

...the tool's own complaint...

Work dir:

/BIODATA/StudentsHome/you/snp_call_test/snp_call_nf/work/3f/8a2c91...

Line	What it tells you
Caused by	Which step, and which sample
Command exit status	127 = tool not found. Anything else = the tool ran and refused
Command error	The tool's own words. Read this before anything else.
Work dir	Where the evidence is

Command error is the line that usually contains the answer. Everything above it is Nextflow explaining that something went wrong. That line is the tool explaining *what*.

5.2b Go to the work directory

Copy the work dir path from the error and go there:

```
cd PASTE_THE_WORK_DIR_PATH
ls -la
```

Everything that task had is here. The inputs, as links. The script. The output it managed to produce before it died.

```
cat .command.err      # what went wrong
cat .command.sh       # exactly what was run
```

Now find the actual cause. GATK complained about a file with the wrong number of fields — look at that file:

```
cat -A gvcf_map.txt
```

✓ **It is empty.** GATK was told to read a list of per-sample variant files, and the list has nothing in it, because those files were never made.

The error message told you the symptom. The work directory told you the cause. That gap is normal, and closing it is the skill.

5.2c Re-run it by hand

You can run the failed step on its own, without the pipeline. Try the obvious thing first:

```
bash .command.sh
```

✓ **You will probably get `gatk: command not found`.** That is expected, and it teaches you something about how Nextflow works.

`.command.sh` contains only the tool command. It does not set up the software environment — that is a separate file:

```
ls .command.*
```

File	What it is
<code>.command.sh</code>	The tool command, and nothing else
<code>.command.run</code>	The wrapper — activates the environment, then calls <code>.command.sh</code>
<code>.command.err</code>	What the tool complained about
<code>.command.out</code>	What the tool printed
<code>.command.log</code>	Both together
<code>.exitcode</code>	The number the tool exited with

Nextflow never runs `.command.sh` directly. It runs `.command.run`, which activates the conda environment first. So that is what you run too:

```
bash .command.run
```

✓ **Now it works** — the same failure as before, reproduced on demand, in seconds, with no pipeline around it.

Or, if you only want the tool command, put the tools on your path first:

```
export PATH=$SNPTRAIN/envs/snp_call_nf/bin:$PATH
bash .command.sh
```

(Only do this in a throwaway terminal — that environment contains an old Java that will stop Nextflow from starting.)

This is how you debug a Nextflow task. You do not debug it through Nextflow. You go to the work directory, reproduce the failure on its own, change things until you understand it, and only then go back and re-run the pipeline.

Run it as many times as you like. Nothing else is affected.

Go back when you are done:

```
cd $HOME/snp_call_test/snp_call_nf
```

5.3 Fix it, and do not start over

The fix is simple. You know what was wrong — that option caused the failure — so take it away. There is nothing clever to repair.

The interesting part is what happens next. You met `-resume` in 2.6, on a stub run where nothing was at stake. This is the version that matters: a real run, a real failure, and real work already finished.

Remove the option, and add `-resume`:

```
nextflow main.nf -c $SNPTRAIN/training.config -resume
```

✓ **Watch the clock.** This run finishes in seconds.

Look at the task table. Every line ends in `cached`, and the summary at the end says nothing succeeded and everything was `cached`.

Nothing ran. Nothing needed to.

What just happened

Before running any task, Nextflow calculates a fingerprint from three things:

1. The script for that step
2. The input files it will receive
3. The parameters in effect

If a task with that exact fingerprint has run before in this directory, Nextflow does not run it again. It reuses the finished result.

When you added `--parasite_reads_only`, the parameters changed, so the fingerprints changed, so the joint-calling steps counted as new work — and that new work failed. Taking the option away restored the original fingerprints, which Nextflow had already seen this afternoon.

That earlier run took about sixteen minutes. You just recovered all of it in about ten seconds.

Why this matters more than it looks

Real runs are not sixteen minutes. They are two days. And they fail — a full disk, a corrupt file, a wrong option, a machine rebooting.

Without `-resume`, every failure costs you everything.

With it, a failure at hour thirty-eight costs you one step.

Two conditions. `-resume` only works if you are in the same directory, and if `work/` is still there. Delete `work/` and the fingerprints have nothing to match against — you start from zero.

That is the trade: `work/` is large and annoying, and it is also the only reason you can recover a long run. Keep it until your results are safe.

If you want a specific run

```
nextflow log
```

Every run you have done, with its name. To continue a particular one rather than the most recent:

```
nextflow main.nf -c $SNPTRAIN/training.config -resume RUN_NAME
```

5.4 The four questions

When a run fails, ask these in order:

1. **Did it fail before starting?** → your sample sheet or your paths
 2. **What is the exit status?** → 127 means the tool was not found; anything else usually means the tool ran and was unhappy
 3. **What does .command.err say?** → the tool's own words, always
 4. **Can I reproduce it with bash .command.run?** → if yes, you can fix it
-

Part 6 — Your own data: start small (15:45)

You will not do this today. This is the plan for next week.

The goal is not to run everything. The goal is to run something successfully, then grow it.

6.1 The three-step ladder

Never point a new pipeline at your whole dataset. Climb:

Step	Samples	What you learn
1	2	Do my paths work? Does my sample sheet parse?
2	~20	How long does one sample take? How much disk? Is my data any good?
3	Everything	Now you can predict the answer before you start

Step 2 is the important one. Twenty samples is big enough to measure and small enough to throw away.

Each step is the same command. Only the sample sheet changes.

6.2 The sample sheet is the control

```
head -3 fastq_map.tsv
```

Five columns, separated by **tabs**:

Column	Name	Meaning
1	Sample	Your name for the sample
2	HostId	Which host genome to filter against — a number, starting at 0
3	Run	Run or replicate identifier, e.g. r1
4	MateId	1 and 2 for paired-end; 0 for single-end
5	Fastq	Full path to the .fastq.gz file

The first line is a header — the column names themselves. Keep it when you make a pilot set, and skip it when you count things.

Paired-end means **two lines per run** — MateId 1 and 2. A sample sequenced twice gets two runs, so four lines. That is why Sample10 produced Sample10~r1 and Sample10~r2 today, and why the early steps ran 6 times for 5 samples.

HostId is an index into a list in the configuration, not a label:

```
grep -n "host" nextflow.config
```

Making a pilot set is just taking fewer lines. Keep whole samples together — never split a sample's two mate lines:

```
head -5 my_samples.tsv > pilot.tsv      # first 2 paired-end runs
```

Check it looks right before running it:

```
tail -n +2 pilot.tsv | cut -f1 | sort -u      # which samples am I about to run?
wc -l pilot.tsv
```

6.3 Always stub first

```
nextflow main.nf -stub-run --fq_map pilot.tsv
```

Ten seconds. Catches every path typo, every wrong column, every missing file — before you spend a single CPU-hour.

This is the habit worth taking away from today.

6.4 Then measure

```
nextflow main.nf -c $SNPTRAIN/training.config --fq_map pilot.tsv
```

When it finishes, three numbers tell you whether step 3 is realistic:

```
nextflow log                # how long it took
du -sh work/                # how much scratch space it needed
du -sh results/             # how much you need to keep
```

Multiply by how many more samples you have. **If 20 samples take 4 hours and 200 GB of scratch, then 200 samples take about 40 hours and 2 TB.** Find that out now, not at hour 38.

Two things that help when the numbers are bad:

- `--gvcf_only` stops after per-sample calling, so you can process samples in batches and joint-call later
- `nextflow clean` frees scratch space between runs

6.5 Getting real data

Published *P. falciparum* data lives in the European Nucleotide Archive. You download FASTQ files by accession, put their paths in column 5, and run.

The pipeline repository includes an `ena_data` directory and an `ena_test` profile for working with data from that archive — worth looking at when you get there.

6.6 Running it somewhere else

On a different machine you will need a different configuration file: how many CPUs a task may use, how much memory, and whether there is a scheduler.

`training.config` is a small example of exactly that. Copy it, change the numbers, and keep it next to your project. **The pipeline does not change. Your config does.**

The full recipe

```
# 0. Start a session that survives disconnection
tmux new -s nf          # or: tmux attach -t nf

# 1. New terminal, every time
source SHARED_TRAINING_DIR/env.sh

# 2. Your copy of the pipeline
cd $HOME/snp_call_test/snp_call_nf

# 3. Write your sample sheet (tabs!)
nano my_samples.tsv

# 4. Make a small pilot
head -5 my_samples.tsv > pilot.tsv

# 5. CHECK THE SHAPE – ten seconds (not your data, see 5.1b)
nextflow main.nf -stub-run --fq_map pilot.tsv
```

```
# 6. Run the pilot for real
nextflow main.nf -c $SNPTRAIN/training.config --fq_map pilot.tsv

# 7. Measure, then scale up
nextflow log ; du -sh work/ results/

# 8. If anything dies: read the error, fix it, then
nextflow main.nf -c $SNPTRAIN/training.config --fq_map pilot.tsv -resume
```

Housekeeping

work/ grows very large. Once results are safe:

```
nextflow log
nextflow clean -f -before RUN_NAME
```

Never while a run is going, and never if you still want -resume to work.

Appendix A — Optional: one option, more parallelism

We may not reach this during the session. It takes two minutes and is worth trying on your own.

The joint-calling step is slow because it works through the genome in chunks. By default the chunks are whole chromosomes. You can make them smaller.

```
nextflow main.nf -stub-run --split intervals
```

✓ **Look at the PARAMETERS banner first.** It now says split: intervals instead of split: chromosomes. The pipeline is confirming your flag.

✓ **Then the four joint-calling steps go from 14 tasks each to 44 each** — and the total at the top climbs from 115 to 235.

Nothing in the pipeline code changed. You did not edit main.nf. You added one word on the command line and the pipeline split the work into more pieces, which means more of the machine is used at once.

Those numbers are not arbitrary. Open nextflow.config and find genome_intervals. It holds two lists: 14 chromosomes, and the same genome cut into 44 smaller pieces. --split chooses which list to use.

Why would you want this? Chromosome 14 is 3.3 Mb; chromosome 1 is 0.64 Mb. Splitting by chromosome means one task takes five times longer than another, and the whole stage waits for the slowest. Sub-chromosomal intervals even that out.

```
grep -n "split" nextflow.config
```

This is the theme of today. You are not going to write Nextflow pipelines. You are going to *run* them, and control them from the outside.

Other options that exist

You will not run all of these, but you should know they are there:

Option	What it does
--split intervals	More parallel chunks in joint calling
--coverage_only	Stop after calculating coverage
--gvcf_only	Stop after per-sample variant calls
--vqsr true	Use machine-learning filtering instead of hard filters
--use_concat_genome	A gentler way of removing human reads
--fq_map FILE	Use a different sample sheet
-output-dir DIR	Write results somewhere other than results/

These options change what the pipeline does without changing its code. Some of them interact in ways that are not obvious — always confirm with a stub run before relying on one.

Command reference card

Command	Purpose
tmux new -s nf	Start a session that survives disconnection
Ctrl-b then d	Detach — leave it running
tmux attach -t nf	Come back to it
source ../env.sh	Turn on Nextflow — every new terminal
nextflow -version	Check it is working
nextflow main.nf -stub-run	Dry run — check the plan, run nothing
nextflow main.nf	Real run, test data
-resume	Continue, reusing finished work
-c FILE	Add a configuration file
--fq_map FILE	Use a different sample sheet
-output-dir DIR	Write results somewhere other than results/
--split intervals	More parallel chunks
--gvcf_only	Stop after per-sample calls
nextflow log	List your previous runs
nextflow clean -f -before RUN	Delete old scratch space
cat .command.sh	The exact command a task ran
bash .command.run	Re-run a failed task by hand (sets up the environment)
cat .command.err	Why a task failed

Words we used

Process — a recipe for one step. Written once, in `main.nf`.

Task — one execution of a process, for one sample or one chunk. Many tasks per process.

Channel — the connection that carries data from one process to the next. The conveyor belt.

Tuple — a group of items travelling together as one unit, in a fixed order. Usually a label plus a file, so the data never loses its name. A row of your sample sheet is a tuple.

Workflow — the block in `main.nf` that wires the processes together.

Executor — where tasks actually run: this machine (`local`), or Slurm, or SGE.

Profile — a named bundle of settings, chosen with `-profile`.

Work directory — the private folder for one task. Where the evidence is.

Stub run — a rehearsal. Full plan, no tools.

After today

Office hour at 16:00, thirty minutes, optional. **Bring your own sample sheet** and we will get it working together.

Pipeline: https://github.com/bguo068/snp_call_nf Nextflow documentation: <https://www.nextflow.io>

Further reading

You will not need all of this. Keep it for when you do.

File formats — the actual specifications - SAM/BAM: <https://samtools.github.io/hts-specs/SAMv1.pdf> - VCF 4.2: <https://samtools.github.io/hts-specs/VCFv4.2.pdf>

Working with these files - samtools: <https://www.htslib.org/doc/samtools.html> - bcftools: <https://samtools.github.io/bcftools/bcftools.html> - A practical walkthrough of sequencing file formats: https://bioinformatics.ccr.cancer.gov/docs/b4/Module2_RNA_Sequencing/

This pipeline and its methods - Pipeline: https://github.com/bguo068/snp_call_nf - MalariaGEN Pf6 extended methods — the source of this pipeline's approach: https://ngs.sanger.ac.uk/production/malaria/pfcommunityproject/Pf6/Pf_6_extended_methods.pdf

Nextflow - Documentation: <https://www.nextflow.io/docs/latest/>